

A Survey and Experimental Evaluation of ExpoSort

Ryan Brooks
George Mason University
Fairfax, Virginia
Brooksryan63@gmail.com

Abstract – ExpoSort is a recently proposed reluctant sorting algorithm designed to sort correctly while doing so as inefficiently as possible. The algorithm recursively sorts the first $(n-1)$ elements of an array, conditionally swaps the last two elements, and then recursively sorts the first $(n-1)$ elements again, producing a theoretical running time of $\Theta(2^n)$. This paper surveys the ExpoSort algorithm, summarizes its theoretical properties, and presents an experimental evaluation of its practical runtime behavior. ExpoSort is compared against CubeSort, InsertionSort, StoogeSort, and SlowSort using Python implementations tested on random, sorted, and reverse-sorted arrays. The experiments measure runtime, function or recursive calls, comparisons, and swaps across 101 trials for each input size and category. The results support the theoretical analysis: ExpoSort exhibits clear exponential growth, reaching more than 16 million recursive calls at $(n=24)$, while input order has little effect on its call and comparison counts. The experiments also confirm the theoretical relationship between ExpoSort and InsertionSort, since both algorithms produce matching swap counts on the tested inputs. Overall, the study shows that ExpoSort’s inefficiency comes primarily from repeated recursive overhead rather than from additional data movement, while CubeSort demonstrates how a small control-flow change can reduce the algorithm from exponentially slow to comparatively practical.

I. INTRODUCTION

SORTING ALGORITHMS are a central topic in the study of algorithms because they provide clear examples of correctness, asymptotic analysis, and practical performance measurement. Most sorting algorithms are designed with efficiency in mind. Algorithms such as MergeSort, HeapSort, QuickSort, and InsertionSort are normally studied to understand how different design strategies affect running time, memory usage, and behavior on different input orders. However, inefficient sorting algorithms can also be useful objects of study. By examining algorithms that are intentionally slow, it becomes easier to observe how

recursive structure, input order, and repeated work affect performance.

ExpoSort is a deliberately inefficient sorting algorithm introduced by Abrahamsen in 2024 [1]. The algorithm recursively sorts the first $(n - 1)$ elements of an array, compares and possibly swaps the final two elements, and then recursively sorts the first $(n - 1)$ elements again. This simple structure produces the recurrence $(T(n) = 2T(n - 1) + \Theta(1))$, which solves to $\Theta(2^n)$. ExpoSort is, therefore, not merely an inefficient implementation of sorting; it is an intentionally exponential sorting algorithm designed within the tradition of pessimal or reluctant algorithms.

Reluctant algorithms are algorithms that delay completion while still appearing to make meaningful progress toward a solution. The concept was formalized by Broder and Stolfi in their work on pessimal algorithms [2]. This distinguishes reluctant algorithms from algorithms that simply waste time through irrelevant loops or artificial delays. Rather than introducing arbitrary delays, a reluctant algorithm continues to work toward the solution while doing so in an intentionally inefficient manner. In the context of sorting, ExpoSort is interesting because it continues to move the array toward sorted order while repeatedly performing unnecessary recursive work. The original ExpoSort paper places the algorithm in relation to earlier inefficient sorting algorithms such as SlowSort, StoogeSort, and BogoSort. Among these, SlowSort is especially important because it was previously known as a major example of a reluctant sorting algorithm. ExpoSort extends this line of work by achieving exponential running time through a very short recursive procedure.

A second reason ExpoSort is worth studying is its close relationship to more practical sorting behavior. The original paper shows that a small modification to ExpoSort produces CubeSort, an algorithm whose second recursive call occurs only after a swap. This minor control-flow change dramatically alters the algorithm’s complexity, giving CubeSort best-case running time $\Theta(n)$ and worst-case running time

$(\theta(n^3))$ [1]. The paper also proves that ExpoSort and CubeSort perform the same swap sequence as InsertionSort. This result is significant because it suggests that ExpoSort’s inefficiency is not caused by extra data movement. Instead, ExpoSort performs insertion-sort-like swaps while surrounding them with exponentially repeated recursive calls.

This paper surveys the ExpoSort algorithm and evaluates whether its theoretical behavior is reflected in experimental results. The study first summarizes the background and theoretical properties of ExpoSort, including its running time recurrence and its relationship to CubeSort and InsertionSort. It then presents a Python-based experimental methodology comparing ExpoSort against CubeSort, InsertionSort, StoogeSort, and SlowSort. The experiments measure runtime, function or recursive calls, comparisons, and swaps on random, sorted, and reverse-sorted arrays. The purpose of these experiments is not only to confirm that ExpoSort is slow, but also to identify why it is slow in practice and whether the observed operation counts match the theoretical claims.

The main contribution of this paper is an empirical evaluation of ExpoSort’s behavior using reproducible test cases. The results are used to compare ExpoSort with both reluctant sorting algorithms and a practical baseline algorithm. In particular, the experiments examine whether ExpoSort’s runtime grows exponentially, whether input order significantly affects its runtime, and whether its swap counts match those of InsertionSort. These observations provide a practical complement to the theoretical analysis in the original ExpoSort paper and help clarify the source of the algorithm’s intentionally poor performance.

II. METHODOLOGY

The experimental portion of this study was designed to compare ExpoSort’s observed behavior with its theoretical analysis and with several related sorting algorithms. All algorithms were implemented in Python and tested on the same generated input cases. The benchmark measured runtime, function or recursive calls, comparisons, and swaps. Each algorithm’s output was also checked against Python’s built-in sorted result to verify correctness before recording the trial.

A. Algorithms Tested

Five sorting algorithms were included in the benchmark: ExpoSort, CubeSort, InsertionSort, StoogeSort, and SlowSort. ExpoSort was the main

subject of the experiment. CubeSort was included because it differs from ExpoSort by only one control-flow change: the second recursive call is made only when a swap occurs. This makes CubeSort an especially useful comparison because the original ExpoSort paper shows that this small change reduces the algorithm from exponential time to polynomial time [1].

InsertionSort was included as a practical baseline. It is not a reluctant sorting algorithm, but it is theoretically important because the original paper proves that ExpoSort and CubeSort perform the same swap sequence as InsertionSort [1]. Comparing ExpoSort with InsertionSort therefore helps distinguish inefficiency caused by data movement from inefficiency caused by recursive overhead. StoogeSort and SlowSort were included as examples of historically relevant inefficient recursive sorting algorithms. SlowSort is especially relevant because the ExpoSort paper presents ExpoSort as surpassing SlowSort’s quasi-polynomial running time in the context of reluctant sorting [1].

B. Test Case Generation

The benchmark used a fixed random seed, 20260616, to make the experiment reproducible. For each input size and trial, the program generated one random permutation of the integers $(0, 1, \dots, n - 1)$. From this base value set, the program created three input categories: random, sorted, and reverse-sorted. The random input was the shuffled permutation. The sorted input was the increasing-order version of the same values, and the reverse input was the decreasing-order version.

The tested input sizes were $n \in \{4, 8, 12, 16, 20, 22, 24\}$. These sizes were chosen because ExpoSort grows exponentially and becomes expensive even for small arrays. Testing through $(n=24)$ was large enough to make the growth trend visible while still allowing the benchmark to complete in a reasonable amount of time.

For each combination of algorithm, input size, and input category, 101 trials were performed. Every algorithm was run on the same generated test cases, so comparisons between algorithms were based on identical inputs. This reduced the possibility that one algorithm would appear faster or slower simply because it received easier or harder random arrays.

C. Metrics Collected

For each trial, the benchmark recorded four main measurements. First, runtime was measured using Python’s `perf_counter_ns()` function and converted to milliseconds. Second, the benchmark counted the number of function or recursive calls. For recursive algorithms, this count increased every time the sorting function was invoked. Since InsertionSort is iterative rather than recursive, its call count was recorded as one per trial. Third, the benchmark counted element comparisons. Fourth, it counted swaps, defined as exchanges of two array elements.

All experiments were conducted using Python 3.14.3 on a personal desktop PC running 64-bit Microsoft Windows 11 Education N. The system used an AMD Ryzen 5 5600G processor with 6 cores and 12 logical processors, and approximately 16 GB of RAM. Because runtime measurements depend on both algorithm behavior and hardware characteristics, the reported execution times should be interpreted relative to this experimental environment.

Both mean and median runtime were computed for each group of 101 trials. Median runtime was used as the main runtime measure because timing tests can be affected by occasional outliers caused by background processes, interpreter overhead, or system scheduling. The mean was also retained in the output data as a secondary measure.

D. Correctness Verification

Each trial included a correctness check. Before running a sorting algorithm, the benchmark computed the expected output using Python’s built-in `sorted()` function. After the tested algorithm finished, the program compared the algorithm’s output with the expected sorted array. If the output did not match, the benchmark raised an error and stopped. This ensured that all recorded performance data came only from successful sorting runs.

E. Data Output and Analysis

The benchmark saved the results to a CSV file. Each row represented one trial and included the trial number, algorithm name, input size, input category, runtime, call count, comparison count, swap count, mean runtime, and median runtime. The full dataset therefore contained one row for each individual run, while also preserving the grouped mean and median runtime values for each algorithm, input size, and input category.

The resulting data were used to compare theoretical and experimental behavior. In particular, the experiments were designed to test whether ExpoSort’s runtime and call count grow exponentially, whether input order significantly affects ExpoSort’s runtime, whether CubeSort behaves differently on sorted and reverse-sorted inputs, and whether ExpoSort’s swap counts match those of InsertionSort. These questions correspond directly to the theoretical properties described in the original ExpoSort paper. The source code, benchmark script, and generated result files are available in the project repository: <https://github.com/Ryanline/exposort-experimentation>.

III. THEORETICAL RESULTS

This section summarizes the main theoretical properties of ExpoSort and the related algorithms most relevant to this study. The central result is that ExpoSort has exponential running time, despite having a short and simple recursive structure. A second important result is that ExpoSort performs the same swap sequence as InsertionSort, which implies that its inefficiency is caused primarily by repeated recursive overhead rather than by additional data movement [1].

A. Correctness of ExpoSort

ExpoSort sorts an array recursively by repeatedly sorting prefixes of the input. For an array A of length n , the algorithm first sorts the first $n - 1$ elements. It then compares the final two elements, $A[n - 1]$ and $A[n]$, and swaps them if they are out of order. Finally, it sorts the first $n - 1$ elements again.

The correctness of ExpoSort follows by induction on n . For $n = 1$, the array is already sorted. Assume that ExpoSort correctly sorts arrays of size $n - 1$. When ExpoSort is applied to an array of size n , the first recursive call sorts the first $n - 1$ elements. If $A[n - 1]$ is less than or equal to $A[n]$, then the largest element of the sorted prefix is already less than or equal to the final element, so the whole array is sorted. If $A[n - 1]$ is greater than $A[n]$, the algorithm swaps the final two elements. After this swap, the final position contains the previous largest element of the sorted prefix, so the final element is in the correct position. The second recursive call then sorts the first $n - 1$ elements. Therefore, after the second recursive call, the entire array is sorted. By induction, ExpoSort correctly sorts arrays of all positive sizes.

B. Running Time of ExpoSort

Let $T(n)$ represent the running time of ExpoSort on an input of size n . For $n > 1$, the algorithm makes two recursive calls on an input of size $n - 1$. Between those calls, it performs one comparison and possibly one swap. The non-recursive work is constant, so the recurrence is

$$T(n) = 2T(n - 1) + \theta(1). \quad (1)$$

Expanding the recurrence gives

$$T(n) = 2(2T(n - 2) + \theta(1)) + \theta(1).$$

This simplifies to

$$T(n) = 2^2T(n - 2) + 3\theta(1).$$

After k expansions, this becomes

$$T(n) = 2^kT(n - k) + (2^k - 1)\theta(1).$$

Setting $k = n - 1$ gives

$$T(n) = 2^{n-1}T(1) + (2^{n-1} - 1)\theta(1).$$

Therefore,

$$T(n) = \theta(2^n). \quad (2)$$

This means that each increase in input size approximately doubles the amount of recursive work performed by ExpoSort. The algorithm is therefore impractical even for moderately small values of n .

C. Recursive Calls and Comparisons

In addition to runtime, ExpoSort's recursive-call count can be analyzed exactly. Let $C(n)$ be the number of times ExpoSort is called while sorting an array of size n , including the initial call. For $n=1$, there is one call, so $C(1)=1$. For $n>1$, the algorithm makes two recursive calls on size $n-1$, plus the current call. Thus,

$$C(n) = 2C(n - 1) + 1. \quad (3)$$

Solving this recurrence gives

$$C(n) = 2^n - 1. \quad (4)$$

The number of comparisons follows a similar recurrence. Let $M(n)$ be the number of element comparisons made by ExpoSort. For $n = 1$, no

comparison is needed, so $M(1) = 0$. For $n > 1$, the algorithm makes two recursive calls on size $n - 1$ and performs one comparison between $A[n - 1]$ and $A[n]$. Therefore,

$$M(n) = 2M(n - 1) + 1. \quad (5)$$

Solving this recurrence gives

$$M(n) = 2^{n-1} - 1. \quad (6)$$

These two results are important because ExpoSort always makes both recursive calls whenever $n > 1$. As a result, the number of calls and comparisons does not depend on whether the input is sorted, reverse-sorted, or randomly ordered. Input order may affect the number of swaps, but it does not change the recursive structure of the algorithm.

D. Swap Behavior and Relationship to InsertionSort

Although ExpoSort has exponential running time, its swap behavior is much less extreme. The original paper proves that ExpoSort and CubeSort perform the same swap sequence as InsertionSort [1]. This means that, on the same input array, ExpoSort moves elements through the same adjacent exchanges that InsertionSort would perform.

For InsertionSort, the number of adjacent swaps is equal to the number of inversions in the input array. An inversion is a pair of elements that appear in the wrong relative order. A sorted array has zero inversions, while a reverse-sorted array of size n has $\frac{n(n-1)}{2}$ inversions. Therefore, ExpoSort also performs zero swaps on sorted input and $\frac{n(n-1)}{2}$ swaps on reverse-sorted input. For a random permutation of n distinct elements, the expected number of inversions is $\frac{n(n-1)}{4}$.

This shows that ExpoSort's swap count is at most quadratic, even though its running time is exponential. Therefore, the algorithm's inefficiency is not caused primarily by moving elements more than necessary. Instead, ExpoSort performs an insertion-sort-like sequence of swaps but reaches those swaps through exponentially repeated recursive calls.

E. CubeSort as a Structural Comparison

CubeSort is a closely related algorithm formed by moving ExpoSort’s second recursive call inside the conditional swap statement. In ExpoSort, the second recursive call is always performed when $n > 1$. In CubeSort, the second recursive call occurs only if the comparison between the final two elements causes a swap.

This small change produces a large difference in asymptotic behavior. The original paper proves that CubeSort has best-case running time $\theta(n)$ and worst-case running time θn^3 [1]. On sorted input, no swaps occur, so CubeSort makes only one recursive call at each level and behaves linearly. On reverse-sorted input, many swaps occur, causing additional recursive work and producing cubic worst-case behavior.

The contrast between ExpoSort and CubeSort is theoretically significant because the two algorithms differ only in one control-flow decision. ExpoSort’s unconditional second recursive call creates exponential growth. CubeSort’s conditional second recursive call prevents this exponential explosion and results in a polynomial-time algorithm. This demonstrates how a minor structural change in a recursive algorithm can greatly affect its asymptotic complexity.

F. Theoretical Expectations for the Experiment

The theoretical analysis leads to several expectations for the experimental results. First, ExpoSort’s runtime and recursive-call count should grow exponentially with input size. Second, ExpoSort’s call and comparison counts should be the same for sorted, random, and reverse-sorted inputs of the same size. Third, ExpoSort and InsertionSort should have matching swap counts on identical inputs. Fourth, CubeSort should perform very well on sorted input but much worse on reverse-sorted input. The experimental section evaluates whether these theoretical expectations appear in the measured results.

IV. EXPERIMENTAL RESULTS

This section presents the results of the Python benchmark described in the methodology section. The experiments compared ExpoSort, CubeSort, InsertionSort, StoogeSort, and SlowSort on random, sorted, and reverse-sorted arrays. For each algorithm, input size, and input category, 101 trials were

performed. Median runtime is emphasized because it better represents typical performance in the presence of occasional timing outliers.

A. Runtime Growth on Random Inputs

Fig. 1 shows the median runtime of all five algorithms on random inputs. The y-axis is logarithmic because ExpoSort grows much more quickly than the other algorithms. Without a logarithmic scale, the non-ExpoSort algorithms would appear nearly flat near zero.

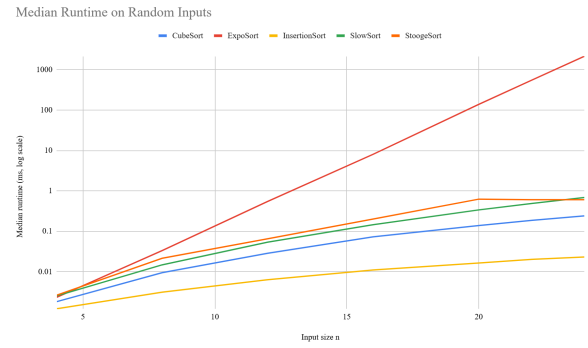


Fig. 1. Median runtime on random inputs across 101 trials. The y-axis is logarithmic because ExpoSort grows much more quickly than the comparison algorithms.

As shown in Fig. 1, ExpoSort’s runtime grows dramatically as input size increases. On random inputs, ExpoSort increased from 0.0023 ms at ($n = 4$) to 2150.1884 ms at ($n = 24$). The growth is especially clear at the larger tested sizes. ExpoSort required 138.5326 ms at ($n = 20$), 548.6665 ms at ($n = 22$), and 2150.1884 ms at ($n = 24$). These increases are consistent with the theoretical expectation that the runtime approximately quadruples when (n) increases by 2, since ExpoSort has exponential ($\theta(2^n)$) running time.

The comparison algorithms remained much faster over the tested input sizes. At ($n = 24$) on random input, CubeSort had a median runtime of 0.2399 ms, InsertionSort had a median runtime of 0.0230 ms, SlowSort had a median runtime of 0.6863 ms, and StoogeSort had a median runtime of 0.6040 ms. In contrast, ExpoSort required 2150.1884 ms. Thus, over the tested range, ExpoSort was several orders of magnitude slower than every comparison algorithm.

B. Effect of Input Order on ExpoSort

The results also show that input order has little effect on ExpoSort's runtime. Sorted, random, and reverse-sorted inputs produced nearly identical recursive-call and comparison counts for the same input size. This matches the theoretical structure of the algorithm. ExpoSort always performs both recursive calls whenever ($n > 1$), regardless of whether the input is already sorted or completely reversed.

At ($n = 24$), ExpoSort made 16,777,215 function calls and 8,388,607 comparisons for each input category. These values match the theoretical call count ($C(n) = 2^{n-1}$) and comparison count ($M(n) = 2^{n-1} - 1$). Therefore, the experimental operation counts directly support the recurrence analysis from the theoretical results section.

Input order did affect the number of swaps. On sorted input, ExpoSort performed zero swaps. On reverse-sorted input at ($n = 24$), it performed 276 swaps, which equals $(\frac{n(n-1)}{2})$ for ($n = 24$). On random input at ($n = 24$), the median swap count was 138. These results are consistent with the idea that swaps depend on the inversion structure of the input, while calls and comparisons depend primarily on ExpoSort's recursive structure.

C. Comparison with InsertionSort

The results support the theoretical relationship between ExpoSort and InsertionSort. The original analysis states that ExpoSort and InsertionSort perform the same swap sequence on the same input [1]. The experimental results matched this claim. For example, at ($n = 24$), both ExpoSort and InsertionSort performed 0 swaps on sorted input, 276 swaps on reverse-sorted input, and a median of 138 swaps on random input.

However, the runtimes were extremely different. On random input at ($n = 24$), InsertionSort had a median runtime of 0.0230 ms, while ExpoSort had a median runtime of 2150.1884 ms. This contrast shows that ExpoSort's inefficiency is not caused by performing more swaps than InsertionSort. Instead, ExpoSort performs the same essential adjacent exchanges while surrounding them with exponentially many recursive calls.

D. CubeSort Results

CubeSort behaved much more like a slower recursive version of InsertionSort than like ExpoSort. This is expected because CubeSort only makes its second recursive call when a swap occurs. On sorted input at ($n = 24$), CubeSort made only 24 calls and 23 comparisons. This reflects its best-case behavior, since no swaps occur and the second recursive call is never triggered.

On reverse-sorted input at ($n = 24$), CubeSort made 2324 calls and 2047 comparisons. This is substantially more work than on sorted input, but still far below ExpoSort's 16,777,215 calls and 8,388,607 comparisons at the same input size. These results support the theoretical claim that moving ExpoSort's second recursive call inside the conditional statement changes the algorithm's behavior dramatically.

E. Summary of Experimental Findings

Overall, the experimental results strongly support the theoretical analysis. ExpoSort exhibited clear exponential growth, while the other algorithms remained much faster over the tested range. ExpoSort's call and comparison counts were independent of input order, confirming that its recursive structure dominates its runtime. Its swap counts matched InsertionSort's swap counts, supporting the claim that ExpoSort performs the same swap sequence as InsertionSort. Finally, CubeSort demonstrated how a small change in control flow can prevent ExpoSort's exponential explosion and produce a much more practical algorithm.

V. DISCUSSION

The experimental results support the theoretical claims made about ExpoSort. The most important finding is that ExpoSort's measured runtime grows extremely quickly as input size increases. This is consistent with the recurrence ($T(n) = 2T(n - 1) + \theta(1)$), which gives ExpoSort a running time of $(\theta(2^n))$ [1]. In the experiment, this behavior was visible even for small values of (n). By ($n = 24$), ExpoSort required more than 16 million function calls and took more than two seconds per run, while the comparison algorithms remained much faster over the same input range.

One notable result is that input order had little effect on ExpoSort's runtime. Sorted, random, and reverse-sorted inputs produced the same number of

calls and comparisons for a fixed input size. This is expected because ExpoSort always makes both recursive calls whenever ($n > 1$). Whether the input is already sorted or completely reversed, the algorithm still recursively sorts the prefix, performs one comparison, and recursively sorts the prefix again. Therefore, the recursive structure dominates the algorithm's runtime. Input order changes the number of swaps, but it does not change the number of recursive calls or comparisons.

The swap counts are especially important because they clarify the source of ExpoSort's inefficiency. The original ExpoSort paper proves that ExpoSort and CubeSort perform the same swap sequence as InsertionSort [1]. The experimental results supported this claim: ExpoSort and InsertionSort had matching swap counts on the tested inputs. For example, both algorithms performed zero swaps on sorted inputs and $\frac{n(n-1)}{2}$ swaps on reverse-sorted inputs.

However, their runtimes were dramatically different. This shows that ExpoSort is not slow because it performs more useful data movement than InsertionSort. Instead, it performs the same adjacent swaps while surrounding them with exponentially many recursive calls.

The comparison with CubeSort further illustrates the importance of control structure. CubeSort differs from ExpoSort only in whether the second recursive call is unconditional or conditional. In CubeSort, the second recursive call occurs only when a swap is made. This small change causes a major difference in behavior. On sorted inputs, CubeSort makes only one recursive call per level and behaves almost linearly. On reverse-sorted inputs, it performs more work, but its operation counts remain far below those of ExpoSort. This agrees with the original paper's claim that CubeSort has $(\theta(n))$ best-case time and $(\theta(n^3))$ worst-case time [1]. In practical terms, CubeSort behaves like a slower recursive version of InsertionSort rather than like an exponential reluctant algorithm.

The results also show that SlowSort and StoogeSort, while intentionally inefficient, were much faster than ExpoSort for the tested input sizes. This does not mean that they are efficient algorithms in a general sense. Rather, it shows that ExpoSort's exponential recurrence causes it to become impractical very quickly. The comparison is useful because it demonstrates that ExpoSort is not merely slower than practical algorithms; it is also much slower than other well-known inefficient sorting algorithms over the tested range.

There are some limitations to the experiment. First, the algorithms were implemented in Python, so the measured runtimes reflect both algorithmic behavior and Python interpreter overhead. In particular, recursive function calls incur nontrivial overhead, which affects recursive algorithms such as ExpoSort, CubeSort, SlowSort, and StoogeSort. Consequently, the reported runtimes should be interpreted as Python-level empirical measurements rather than language-independent execution costs. Second, because ExpoSort grows exponentially, the experiment could only test relatively small input sizes. Larger inputs would quickly become impractical. Third, the runtime measurements were taken on a normal computer system, so they may be affected by background processes and system scheduling. To reduce this issue, the experiment used 101 trials and emphasized median runtime rather than relying only on the mean.

Despite these limitations, the operation counts provide strong support for the theoretical analysis because they are not affected by timer noise in the same way runtime is. ExpoSort's call and comparison counts exactly followed the predicted exponential pattern, and its swap counts matched InsertionSort's. Therefore, even if the exact runtime values would vary on a different machine or implementation, the main conclusion would remain the same: ExpoSort's inefficiency comes primarily from its recursive control structure rather than from the number of swaps it performs.

Overall, the experiment confirms the main theoretical story of ExpoSort. ExpoSort is an intentionally inefficient sorting algorithm whose exponential behavior is easy to observe experimentally. CubeSort shows that a small structural modification can greatly reduce the cost, while InsertionSort shows that the same swap sequence can be performed much more directly. Together, these results demonstrate how algorithmic control flow can dominate practical performance even when the underlying data movement is identical.

VI. CONCLUSION

This paper surveyed ExpoSort and evaluated its behavior experimentally. ExpoSort is a reluctant sorting algorithm that correctly sorts an array while doing so in exponential time. Its recurrence, $(T(n) = 2T(n - 1) + \theta(1))$, predicts $(\theta(2^n))$ running time, and the experimental results strongly supported this analysis. As input size increased, ExpoSort's runtime, recursive-call count, and

comparison count grew rapidly, while input order had little effect on those values.

The experiments also supported the original paper’s claim that ExpoSort performs the same swap sequence as InsertionSort. This was an important result because it showed that ExpoSort’s inefficiency is not caused by extra swapping. Instead, ExpoSort performs insertion-sort-like data movement through an exponentially inefficient recursive structure. The comparison with CubeSort reinforced this conclusion by showing that a single change in control flow can reduce the algorithm from exponential behavior to a much more practical polynomial-time variant.

In summary, ExpoSort provides a useful example of how a simple recursive algorithm can be correct, structured, and steadily progressing while still being intentionally impractical. The experimental results matched the theoretical expectations and showed that ExpoSort’s main cost comes from repeated recursive overhead rather than from additional data movement. As a result, ExpoSort is valuable not as a practical sorting method, but as a clear demonstration of how algorithm design choices affect asymptotic and observed performance.

VII. REFERENCES

- [1] M. Abrahamsen, “ExpoSort: Breaking the quasi-polynomial-time barrier for reluctant sorting,” arXiv:2409.00794, Sep. 2024.
- [2] A. Z. Broder and J. Stolfi, “Pessimal algorithms and simplicity analysis,” SIGACT News, vol. 16, no. 3, pp. 49–53, 1984.
- [3] J. Bentley, “Programming pearls: How to sort,” Commun. ACM, vol. 27, no. 4, pp. 287–ff., 1984.
- [4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [5] H. Gruber, M. Holzer, and O. Ruepp, “Sorting the slow way: An analysis of perversely awful randomized sorting algorithms,” in Fun with Algorithms, 2007, pp. 183–197.